EDGES

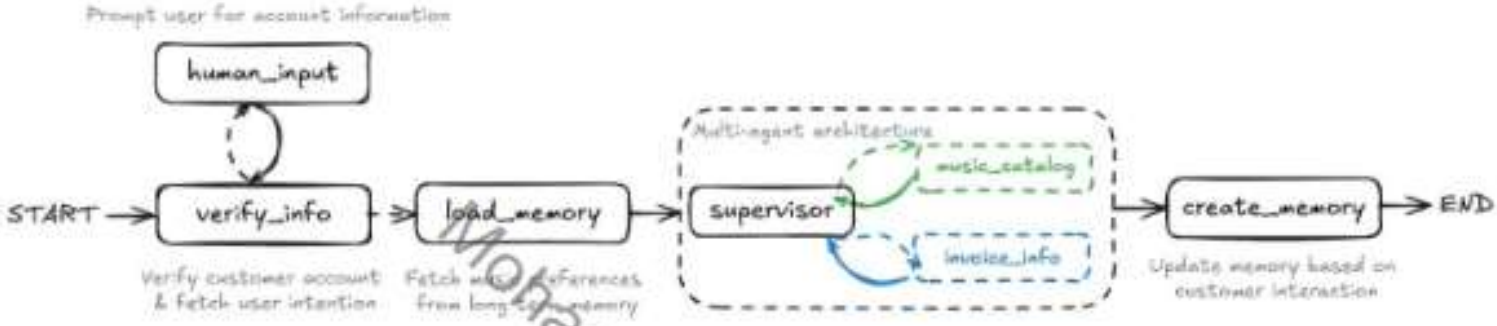
OUR EDGE IS YOUR SUCCESS

Mohamed Hisham, mohamedhisham
mohamedhisham.h.ai@gmail.com

Music Store Customer Support

1. Project Description:

This project is designed to build a Customer Support Multi-Agent System for a digital music store using Python, LangChain, LangGraph, and Jupyter Notebook. The system will simulate a real-world customer support workflow, including:



- **Multi-Agent Architecture:**
  The system uses multiple specialized agents (sub-agents) to handle different types of customer queries, such as music catalog questions and invoice information.
- **Customer Verification:**
  Before answering sensitive queries, the system verifies the customer's identity using their customer ID, email, or phone number, with a human-in-the-loop process for missing or incorrect information.
- **Music Catalog Sub-Agent:**
  Handles queries related to music discovery, recommendations, and catalog search (albums, tracks, genres).
- **Invoice Information Sub-Agent:**
  Handles queries about past purchases, invoice details, and employee support information.
- **Supervisor Agent:**
  Orchestrates the workflow, routing queries to the appropriate sub-agent(s) based on the customer's request.
- **Long-Term Memory:**
  Stores and retrieves user preferences (e.g., favorite genres or artists) to personalize recommendations.
- **Database Integration:**
  Uses an SQLite version of the Chinook music store database for realistic data.
- **Human-in-the-Loop:**
  Interrupts the workflow to request additional information from the user when needed, simulating real customer support escalation.

mohamedhisham mohamedhisham mohamedhisham.ai@gmail.com

---

2. Tasks to Complete the Project:

1. Set Up the Chinook SQLite Database: (Code Provided)

- Download and initialize the Chinook database in-memory using Python.

- Ensure tables such as Customer, Invoice, Artist, Album, Track, and Employee are accessible.
- The database is provided within the url, this cell could be used directly without extra effort

```
import sqlite3 # Standard Python library for SQLite database interaction
import requests # Library for making HTTP requests (to download the SQL script)
from langchain_community.utilities.sql_database import SQLDatabase # LangChain utility
to interact with SQL databases
from sqlalchemy import create_engine # SQLAlchemy function to create a database engine
from sqlalchemy.pool import StaticPool # SQLAlchemy connection pool class for in-memory
databases

def get_engine_for_chinook_db():
    """Pull sql file, populate in-memory database, and create engine."""
    # URL to the raw SQL script for the Chinook database
    url = "https://raw.githubusercontent.com/lerocha/chinook-
database/master/ChinookDatabase/DataSources/Chinook_Sqlite.sql"

    # Fetch the SQL script content from the URL
    response = requests.get(url)
    sql_script = response.text

    # Create an in-memory SQLite database connection.
    # `check_same_thread=False` is important for SQLAlchemy's StaticPool.
    connection = sqlite3.connect(":memory:", check_same_thread=False)

    # Execute the SQL script to populate the in-memory database with Chinook data
    connection.executescript(sql_script)

    # Create a SQLAlchemy engine for the in-memory SQLite database.
    # `creator=lambda: connection` tells SQLAlchemy how to get a new connection.
    # `poolclass=StaticPool` is used for in-memory databases, ensuring the same
connection is reused.
    # `connect_args` are passed directly to the `sqlite3.connect` function.
    return create_engine(
        "sqlite://",
        creator=lambda: connection,
        poolclass=StaticPool,
        connect_args={"check_same_thread": False},
    )

# Get the SQLAlchemy engine for our Chinook database
engine = get_engine_for_chinook_db()
# Create a LangChain SQLDatabase utility instance from the engine.
# This utility will help our agents interact with the database via SQL queries.
db = SQLDatabase(engine)
```

Mohammed Hisham, mohammedhisham.h.ai@gmail.com

2. Define and follow the state schema for your graph:

- **Use the following State Schema**

```
from typing_extensions import TypedDict # For defining dictionaries with type hints
from typing import Annotated, List # For type hinting lists and adding annotations
from langgraph.graph.message import AnyMessage, add_messages # For managing messages in
the graph state
from langgraph.managed.is_last_step import RemainingSteps # For tracking recursion
limits

class State(TypedDict):
    """Represents the state of our LangGraph agent."""
    # customer_id: Stores the unique identifier for the current customer.
    customer_id: str

    # messages: A list of messages that form the conversation history.
    # Annotated with `add_messages` to ensure new messages are appended rather than
overwritten.
    messages: Annotated[list[AnyMessage], add_messages]

    # loaded_memory: Stores information loaded from the long-term memory store,
    # typically user preferences or historical context.
    loaded_memory: str
```

3. Implement Sub-Agents:

Define Clearly prompt for each agent in your system, which describe the role of each agent

- **Music Catalog Sub-Agent:**
    - Create tools/functions to search for albums, tracks, and genres.
        - **def get_albums_by_artist(artist: str):**
            - Retrieves albums by a given artist.
        - **def get_tracks_by_artist(artist: str):**
            - Retrieves tracks (songs) by a given artist or similar artists.
        - **def get_songs_by_genre(genre: str):**
            - Fetches songs that match a specific genre.
        - **def check_for_songs(song_title):**
            - Checks if a song exists by its name.

Mohamedhisham.h.ai@gmail.com Mohamedhisham

- Respond to music-related queries and provide recommendations.

- **Invoice Information Sub-Agent:**
  - Create tools/functions to retrieve invoices, sort by date or price, and find associated employees.
    - **def get_invoices_by_customer_sorted_by_date(customer_id: str) -> list[dict]:**
      - Retrieves all invoices for a customer, sorted by invoice date (most recent first).
    - **def get_invoices_sorted_by_unit_price(customer_id: str) -> list[dict]:**
      - Retrieves all invoices for a customer, sorted by unit price (highest to lowest).
    - **def get_employee_by_invoice_and_customer(invoice_id: str, customer_id: str) -> dict:**
      - Retrieves the employee information associated with a specific invoice and customer.
  - Respond to invoice-related queries.

4. Build the Supervisor Agent:

- Implement a supervisor that analyzes incoming queries and routes them to the correct sub-agent(s).
- Ensure the supervisor can handle multi-step queries that require both music and invoice information.

5. Add Customer Verification:

- Implement a verification node that parses user input for **customer ID, or email, or phone.**
  - **Knowing that if Phone or email are used in the user input, those will be finally mapped to customer ID based on understanding of database**
- If verification fails or information is missing, interrupt the workflow and prompt the user for correct details.

6. Integrate Long-Term Memory:

- Store and retrieve user music preferences using an in-memory store.
- Update preferences based on conversation history.

7. Implement Human-in-the-Loop Workflow:

- Use interrupt/resume logic to handle cases where human input is required. For instance:
  - If user input is correctly interpreted to subsequent sub-agent, but there is missing information, then human input should be required to fill the missing information
  - If user input failed to be interpreted correctly, then human input should be required to entering valid queries
- Short memory should be used to store some useful information like customer id throughout the whole session

Mohamed Hisham Mohamedhisham
mohamedhisham.h.a@gmail.com

- Ensure the system can resume the workflow after receiving new information.

8. Orchestrate the Complete Multi-Agent Workflow:

- Build a state graph that connects all nodes: verification, memory loading, supervisor, memory saving, and human input.
- Ensure the workflow follows the correct sequence and handles edge cases.

9. Final Testing and Debugging:

- Test the system with various customer queries (music, invoices, mixed).
- Simulate missing or incorrect customer information and verify human-in-the-loop handling.
- Ensure user preferences are correctly stored and retrieved.

Deliverables:

1. Recorded Demo of the Running Project:

- A video demonstration:
    - Customer verification (with and without correct info).
    - Music catalog queries and recommendations.
    - Invoice information retrieval.
    - Supervisor routing for multi-step queries.
    - Human-in-the-loop interruptions and resumption.
    - Long-term memory updates and personalized responses.

2. ZIP Folder with Source Code:

- A zipped folder containing:
    - Jupyter Notebook with all code and explanations.

Mohamed Hisham mohamedhisham
mohamedhisham.h.ai@gmail.com

Test Scenarios

1. Testcase 1 “Providing customer information”:

- Input:

```
question = "My phone number is +55 (12) 3923-5555. How much was my most recent purchase? What albums do you have by the Rolling Stones?"
```

- Expected Output:

```
I've found the information you requested:

- Your most recent purchase was on August 7, 2025, for a total amount of $8.91.

Also, here are some albums by The Rolling Stones available in our catalog:

1. **Hot Rocks, 1964-1971 (Disc 1)**
2. **No Security**
3. **Voodoo Lounge**

If you need any more details about these albums or have any other questions, feel free to ask!
```

- Using same session, next input will be:

```
question = "List some songs that match my preferences?"
```

- The output should have songs that belong to Rolling Stones

Mohamed Hisham, mohamedhisham
mohamedhisham.h.ai@gmail.com

2. Test case 2 "Missing Customer information in new session":

- Input:

```
question = "How much was my most recent purchase? What albums do you have by the Rolling Stones?"
```

- Expected Output:
    - Flow will be interrupted, asking Human for right credentials "Customer ID, or Phone, or email"

Mohamed Hisham, mohamedhisham
mohamedhisham.h.ai@

Thank You
Edges For Training Team